Syntactic Conditions for Antichain Property in Consistency Restoring Prolog

Vu H. N. Phan
Rice University (2017-2023)
Texas Tech University (2014-2017)
https://vuphan314.github.io/

2018-07-18 (University of Oxford, England)
Workshop on Answer Set Programming and Other Computing Paradigms
affiliated with International Conference on Logic Programming
part of Federated Logic Conference

Contents

1. Introduction
2. Preliminaries
   - A-Prolog
   - CR-Prolog
3. Results
   - Antichain Property
   - Dependency Graphs
   - Main Antichain Guarantee
4. Conclusion

Section 1

Introduction

Introduction

Logic programming languages:

1. Answer Set Prolog (A-Prolog): standard
   {[1] Gelfond and Lifschitz 1988 "The Stable Model Semantics for Logic Programming"}
2. Consistency Restoring Prolog (CR-Prolog): extension with cr-rules (for rare exceptions)
   {[2] Balduccini and Gelfond 2003 "Logic Programs with Consistency-Restoring Rules"}

Motivation

Informal semantics:

1. Program: specification for answer sets
2. Answer set: beliefs
3. Rationality principle: fewer beliefs are better
4. Antichain property: no answer set is proper subset of another

Contribution

Syntactic condition guaranteeing antichain property:

1. acyclicity (of dependency graph), and
2. cr-independence (no directed path from one cr-rule head literal to another)

Section 2

Preliminaries

Preliminaries

Syntax & semantics:

1. A-Prolog (Answer Set Prolog)
2. CR-Prolog (Consistency Restoring Prolog)

{[3] Gelfond and Kahl 2014 *Knowledge Representation, Reasoning, and the Design of Intelligent Agents: the Answer Set Programming Approach*}

A-Prolog Syntax

1. **Atom**
2. **Literal**: atom a, or its **classical-negation** $\neg a$
3. **Extended literal**: literal l, or its **default-negation** `not` l
4. Literal l:
   1. **appears positive** in extended literal l
   2. **appears negative** in extended literal `not` l

A-Prolog Syntax

1. **Rule**:

$$l_1 \texttt{ or } \ldots \texttt{ or } l_k \longleftarrow l_{k+1}, \ldots, l_m, \texttt{ not } l_{m+1}, \ldots, \texttt{ not } l_n. \qquad (r)$$

2. **Head**: literals before $\longleftarrow$

$$l_0 \longleftarrow l_1, \ldots, l_m, \texttt{ not } l_{m+1}, \ldots, \texttt{ not } l_n. \qquad (\textbf{nondisjunctive rule})$$

3. **Premise**: extended literal after $\longleftarrow$
4. **Body**: premises
5. **Positive body**: literals appearing positive in body

$$l_1 \texttt{ or } \ldots \texttt{ or } l_k \longleftarrow l_{k+1}, \ldots, l_m. \qquad (\textbf{default-negation-free rule})$$

6. **Program**: rules

A-Prolog Semantics

1. **Context**: subset of literals in program
2. **Complementary literals**: a and $\neg a$
3. **Consistent context**: no complementary literals
4. *Convention:* contexts are consistent (from now on)

A-Prolog Semantics

1. Rule:

$$l_1 \texttt{ or } \ldots \texttt{ or } l_k \longleftarrow l_{k+1}, \ldots, l_m, \texttt{ not } l_{m+1}, \ldots, \texttt{ not } l_n.$$

2. Context S **satisfies**:
   1. literal l if: $l \in S$
   2. extended literal $\texttt{not } l$ if: $l \notin S$
   3. rule head if: S satisfies some literal
   4. rule body if: S satisfies all extended literals
   5. rule if: S satisfies head whenever S satisfies body
   6. program if: S satisfies all rules

A-Prolog Semantics

1. Default-negation-free rule:

$$l_1 \texttt{ or } \ldots \texttt{ or } l_k \longleftarrow l_{k+1}, \ldots, l_m.$$

2. Context S is **answer set** of default-negation-free program Π if:
   1. S satisfies Π, and
   2. S is minimal wrt set inclusion (no proper subset of S satisfies Π)

A-Prolog Semantics

1. Rule:
$$l_1 \texttt{ or } \ldots \texttt{ or } l_k \longleftarrow l_{k+1}, \ldots, l_m, \texttt{ not } l_{m+1}, \ldots, \texttt{ not } l_n.$$
2. **Reduct** Π^S of program Π wrt context S: default-negation-free program constructed by
   1. deleting rules containing extended literal `not` l where literal $l \in S$, then
   2. deleting extended literals containing `not`
3. S is **answer set** of Π if: S is answer set of Π^S
4. **Consistent program**: having answer set

CR-Prolog Syntax

1. A-Prolog regular rule:

$$l_1 \texttt{ or } \ldots \texttt{ or } l_k \longleftarrow l_{k+1}, \ldots, l_m, \texttt{ not } l_{m+1}, \ldots, \texttt{ not } l_n.$$

2. CR-Prolog **cr-rule** (consistency restoring rule):

$$l_0 \xleftarrow{+} l_1, \ldots, l_m, \texttt{ not } l_{m+1}, \ldots, \texttt{ not } l_n. \qquad (r)$$

   1. **Cr-literal**: l_0
   2. **Application** $\alpha(r)$: regular rule constructed by replacing $\xleftarrow{+}$ with $\longleftarrow$

3. **CR-Prolog program** $\Pi = \Pi^{reg} \cup \Pi^{cr}$:
   1. **Regular subprogram** Π^{reg}: regular rules
   2. **Cr-subprogram** Π^{cr}: cr-rules

CR-Prolog Semantics

1. CR-Prolog program $\Pi = \Pi^{reg} \cup \Pi^{cr}$
2. $R \subseteq \Pi^{cr}$: subset of cr-rules
3. Application $\alpha(R)$: regular rules
4. $\Pi_R = \Pi^{reg} \cup \alpha(R)$: A-Prolog program under R-application
5. R is **abductive support** of Π if:
   1. Π_R is consistent, and
   2. R is minimal wrt cardinality: no $R' \subseteq \Pi^{cr}$ where $|R'| < |R|$ such that $\Pi_{R'} = \Pi^{reg} \cup \alpha(R')$ is consistent
6. Context S is **answer set** of Π if: S is answer set of Π_R for some abductive support R of Π

CR-Prolog Semantics

Example: hypothetical complexity result

1. CR-Prolog program:

$$p_eq_np \longleftarrow sat_p. \qquad (P = NP \text{ if } SAT \in P)$$

$$-p_eq_np \longleftarrow \texttt{not}\ surprise. \qquad (P \neq NP \text{ unless there is a surprise})$$

$$sat_p \longleftarrow . \qquad (\text{hypothetically } SAT \in P)$$

$$surprise \overset{+}{\longleftarrow} . \qquad (\text{a surprise is unlikely})$$

2. Answer set: $\{sat_p, p_eq_np, surprise\}$

Section 3

Results

Results

1. Antichain property: desirable semantic feature
2. Dependency graphs: syntactic abstractions of programs
3. Main antichain guarantee: acyclicity & cr-independence

Antichain Property

Antichain property: no answer set is proper subset of another

1. A-Prolog programs: always
2. CR-Prolog programs: sometimes

Antichain Property

Example: CR-Prolog program without antichain property

$$a \longleftarrow . \tag{1}$$

$$\neg a \longleftarrow \texttt{not}\ b,\ \texttt{not}\ c. \tag{2}$$

$$b \longleftarrow c. \tag{3}$$

$$b \overset{+}{\longleftarrow} . \tag{4}$$

$$c \overset{+}{\longleftarrow} . \tag{5}$$

1. Answer set chain: $S_1 = \{a, b\} \subsetneq \{a, b, c\} = S_2$
2. Corresponding abductive supports: $R_1 = \{(4)\}$ and $R_2 = \{(5)\}$
3. In (3): "dependence" of cr-literal b from (4) on cr-literal c from (5)

Dependency Graphs

1. CR-Prolog program Π:

$$l_1 \texttt{ or } \ldots \texttt{ or } l_k \longleftarrow l_{k+1}, \ldots, l_m, \texttt{ not } l_{m+1}, \ldots, \texttt{ not } l_n.$$

2. **Dependency graph** G_Π:
   1. Vertices: $l_1, \ldots, l_n$
   2. Directed edges: to head from positive body of each rule

$$(l_{k+1}, l_1), \ldots, (l_m, l_1)$$
$$\vdots$$
$$(l_{k+1}, l_k), \ldots, (l_m, l_k)$$

Acyclic Dependency Graphs

1. CR-Prolog program Π is **acyclic** if: dependency graph G_Π contains no directed cycle
2. *Remark:* context S is answer set of acyclic A-Prolog program Π if
   1. S satisfies Π, and
   2. for each literal $l \in S$, some rule $r \in \Pi$ exists where:
      1. S satisfies $\mathtt{body}(r)$
      2. $\mathtt{head}(r) \cap S = \{l\}$

{[4] Ben-Eliyahu and Dechter 1994 "Propositional Semantics for Disjunctive Logic Programs"}

CR-Independent Dependency Graphs

1. Π: CR-Prolog program
2. G_Π: dependency graph
3. Literal l **depends** on literal l' in Π if: G_Π has directed path to l from l'
4. Π is **cr-independent** if: no cr-literal depends on another
5. *Example:* cr-dependent program

$$a \longleftarrow . \tag{1}$$

$$\neg a \longleftarrow \texttt{not}\ b,\ \texttt{not}\ c. \tag{2}$$

$$b \longleftarrow c. \tag{3}$$

$$b \xleftarrow{+} . \tag{4}$$

$$c \xleftarrow{+} . \tag{5}$$

Answer set chain: $S_1 = \{a, b\} \subsetneq \{a, b, c\} = S_2$

Main Antichain Guarantee

Theorem

All acyclic cr-independent CR-Prolog programs have antichain property (no answer set is proper subset of another).

Section 4

Conclusion

Summary

1. CR-Prolog: A-Prolog extended with consistency restoring rules
2. Desirable antichain property: no answer set is proper subset of another
3. Sufficient syntactic condition:
    1. acyclicity, and
    2. cr-independence